

41039 Programming 1

Week 1: Introduction — Your First Program



Dr. Xuan-Bach Le

`le.xuan.bach@hcmut.edu.vn`

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

Outline

- 1 What Programming Is
- 2 Your First Java Program
- 3 Variables, Types and Values
- 4 The Same Ideas in Python
- 5 When Things Go Wrong
- 6 Ed, Labs and Getting Help

Before we start

- We follow the **UTS course format** for 41039 Programming 1, as conducted at UTS by **Dr. Luke Mathieson**
- Some details may change to suit our actual situation here
- Everything today comes from the **Week 1 lessons on Ed**

Today we assume you have never programmed before.

- If you have, the first part will be a breeze
- If you have not, that is exactly who this subject is designed for

You will see more things today than we can fully explain today. That is on purpose.

Outline

- 1 What Programming Is
- 2 Your First Java Program
- 3 Variables, Types and Values
- 4 The Same Ideas in Python
- 5 When Things Go Wrong
- 6 Ed, Labs and Getting Help

Why are we here?

- Programming is the core practical discipline of computer science and IT
- Almost everything relies on it — food production, logistics, entertainment
- You are probably in a degree that will demand a lot of programming from you

Can't AI just do all this now?

- It is a very useful tool
- But to use it properly, **you need to know how to program without it**

Most people find the concept of programming obvious, but the doing impossible. — Alan Perlis

“Can’t AI just do all this now?”

IN THE NEWS

AI made experienced developers 19% slower

METR randomised controlled trial, July 2025

They predicted they would be **24% faster**.

They were **19% slower**.

They still believed they were **20% faster**.



xkcd 1838, *Machine Learning* — Randall Munroe, CC BY-NC 2.5 — xkcd.com/1838

They could not tell. Neither can you — yet.

What is a program?

At the simplest level, programming is **telling a computer what to do**.

- Once, that meant writing the right sequence of zeroes and ones into memory
- Then coded instructions — simpler, but still explicitly moving memory around
- **1954:** John Backus develops FORTRAN — now the programmer can care more about *what* to do than exactly *how*

Programming language design did not stop in 1954. There are now thousands of languages, with hundreds in active use.

The history of programming is the history of raising the level of abstraction.

The hard part is knowing what is hard



xkcd 1425, *Tasks* — Randall Munroe, CC BY-NC 2.5 — xkcd.com/1425

Two requests.

Both sound reasonable.

One takes an afternoon.

One took the field twenty years.

Telling them apart is a skill. You will not have it this week.

“It’s just a small change” is the most expensive sentence in software.

Readability is relative

FORTRAN code that generates the subsets of a set of size n :

```
SUBROUTINE NEXSUB(N, IN, MTC, NCARD, J)
  INTEGER IN(N)
  LOGICAL MTC
  IF(MTC) GO TO 20
  DO 11 I = 1, N
11  IN(I)=0
  NCARD=0
  MTC=.TRUE.
  RETURN
```

So much easier, right?

Nobody here can read this yet. Neither could anyone on day one of 1954.

The three basic tools of programming

Underlying every general-purpose language are three core concepts:

- **State** — a way of managing what situation the program is in
- **Branching** — making choices about which code to execute, usually based on the state
- **Repetition** — repeating a piece of code, controlled by some parameter

Add basic input and output — reading files, writing to the screen — and that is *everything* you need to build a general-purpose programming language.

This is SLO1: state, iteration and decision.

Where you will meet them

Concept	Week	Topic
State	Week 2	Data Types and Variables
Branching	Week 3	Decisions and Branching Program Flow
Repetition	Week 5	Loops and Iterative Program Flow

Everything from Week 6 onward — classes, methods, lists, files, inheritance — adds **no fourth idea**. It is about **organising** these three well as programs get bigger.

The central difference between languages is how they implement these three.

Programming paradigms

Imperative

State is expressed through variables. The program says *how* to manipulate them.

- **Procedural** — broken into procedures, each a mini-program. *Python originated here.*
- **Object oriented** — state encapsulated in objects along with the code that manipulates it. *Java is here.*

In practice few modern languages sit in a single box. Java can be used procedurally and functionally; Python supports OO and functional styles.

Declarative

Emphasises *what* is computed, not how. State is more ephemeral.

- **Functional** — computation as the application of functions; heavy use of recursion

This is vocabulary for later, not something you need to use this week.

Syntax, semantics, types

- **Syntax** — how the language lets you express intent, grammatically
- **Semantics** — the rules about what is actually meaningful
- **Type systems** — how the language handles data of different kinds

The same idea — “store 42 in a variable called `i`” — in three languages:

```
int i = 42;           // Java
i = 42                # Python
var i : Int = 42;     // Kotlin
```

What did Java make you say that Python did not? The type.

Outline

- 1 What Programming Is
- 2 Your First Java Program
- 3 Variables, Types and Values
- 4 The Same Ideas in Python
- 5 When Things Go Wrong
- 6 Ed, Labs and Getting Help

Java, the language

Java is a general purpose language that is primarily **class-based object oriented**, but allows functional and procedural programming. It is syntactically derived from C, is **strongly statically typed** and has **automatic garbage collection**.

Two of those matter today:

- **Statically typed** — types are fixed and checked before the program runs
- **Garbage collected** — you never free memory by hand. Some languages make you.

Come back to this paragraph in Week 12. You will understand every word.

How Java code is structured

A Java program is a sequence of **statements** arranged into **blocks**. Blocks nest, and the top level is a **class**.

- **Class** — a definition of every object of that sort
- **Method** — a named block of code that may take inputs and return a value
- **Variable** — a named piece of memory holding data, with a **type**

Two sorts of variable

- **Primitives** — simply store a value
- **Objects** — structures combining data with methods that operate on it

Remember this split. It comes back later today.

Conventions — and one Rule

Conventions (not enforced, but everyone follows them)

- Class names start uppercase: HelloWorld, not hello_world
- Variable names start lowercase: myNumber, not MyNumber
- Multi-word names capitalise each word after the first
- Constants are all caps with underscores: Math.PI, Integer.MAX_VALUE
- Indent every time you start a new block

The one Rule you must know

Java is case sensitive.

bob $\neq$ Bob $\neq$ BOB $\neq$ BoB $\neq$ boB

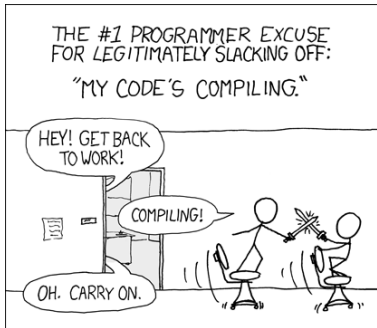
This will be the cause of most of your errors for the next two weeks.

Hello, Java

```
public class Hello {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello 41039!");  
  
    }  
}
```

- `public class Hello` — defines a class named `Hello`
- Its contents run from the opening `{` to the closing `}`
- `main` is a **method**: `<return type> <name>(<parameters>) { ... }`
- `void` means it returns nothing
- Every statement ends with a `;`

An aside: Java is compiled



The whole file is translated **before** anything runs.

- Some mistakes get caught before the program starts
- Python skips this step — a trade, not a free win

xkcd 303, *Compiling* — Randall Munroe, CC BY-NC 2.5 — xkcd.com/303

Your programs compile in under a second. No excuse for the sword fight, sorry.

What you are allowed not to understand yet

Token	What it means	Explained properly in
<code>public</code>	everything can access this	Week 7
<code>static</code>	needs no object to run	Week 7
<code>void</code>	returns nothing	Week 7
<code>String[] args</code>	an array of Strings	Week 4

For a piece of Java code to be runnable, it needs a method with **exactly** this signature:

```
public static void main(String[] args)
```

For now: copy it exactly. Execution always starts there.

One more rule: the file name

The file name must match the public class name.

Class	File
<code>public class Hello</code>	<code>Hello.java</code>
<code>public class HelloWorld</code>	<code>HelloWorld.java</code>

Ed manages files for you, so you may not notice this at first. You will notice it the moment you try to compile on your own machine.

And remember: case sensitive. `hello.java` will not do.

You may also see this

As of Java 25, very simple programs can be written in a simplified form:

```
void main() {  
    IO.println("Hello 41039!");  
}
```

- Same program, less boilerplate
- These features are new, and come with limitations once programs get bigger
- Most code you meet — and everything in this subject — uses the verbose form

Recognise it. Do not use it this session.

It is not just you

IN THE NEWS

Java 25 adds a “smooth on-ramp” so beginners can skip the ceremony

JEP 512, OpenJDK — on `public static void main(String[] args)`

“Mysterious and unhelpful ... not used in small programs.”

If that line felt like meaningless ritual, that was not you being slow.

The people who maintain Java wrote it down as a problem.

We still teach the long form — every codebase you will read uses it.

Outline

- 1 What Programming Is
- 2 Your First Java Program
- 3 Variables, Types and Values
- 4 The Same Ideas in Python
- 5 When Things Go Wrong
- 6 Ed, Labs and Getting Help

Java's primitive types

Type	Size	Stores
byte	8-bit	whole numbers, -128 to 127
short	16-bit	whole numbers, $-32,768$ to $32,767$
int	32-bit	whole numbers, -2^{31} to $2^{31} - 1$
long	64-bit	whole numbers, -2^{63} to $2^{63} - 1$
float	32-bit	decimals, limited accuracy
double	64-bit	decimals, more precise
boolean	—	true or false
char	16-bit	a single Unicode character

In practice you will mostly use int, double, char and boolean.

This number once made global news

2,147,483,647

Gangnam Style views, YouTube, December 2014

That is $2^{31} - 1$ — the largest number an `int` can hold.

YouTube moved the counter to 64 bits: a `long`.

<code>int</code>	about 2.1 billion
<code>long</code>	about 9.2 quintillion

Every type is a promise about how big the number can get.

Declaration, initialisation, assignment

```
int a;           // declaration
a = 1;           // assignment (and initialisation)

boolean b = true; // declaration + initialisation

b = false;       // assignment only

System.out.println("a = " + a);
System.out.println("b = " + b);
```

- **Declaration** — a type followed by a name
- **Initialisation** — the *first* assignment of a value
- **Assignment** — = takes the value on the right, stores it on the left

These three words appear in exercise instructions. Learn them now.

Arithmetic

```
int a = 8;
int b = 7;

int c = a * b;           // 56
System.out.println("c = " + c);

int d = c % (b - 1);      // remainder of 56 / 6 -> 2
System.out.println("d = " + d);
```

- + - * / and % (remainder)
- **Careful:** 7 / 2 is 3, not 3.5. Integer divided by integer gives an integer. Use 7.0 / 2 to get 3.5.

Notice the + in "c = " + c is not addition. It joins Strings together.

A note about =

In maths, $x = 4$ *asserts* that x and 4 are the same thing.

In Java, `x = 4;` **takes the value on the right and stores it in the variable on the left.**

So what does this mean?

$$a = a + 1;$$

As maths it is false for every a . As Java it is completely routine: *take $a + 1$, put it in a .*

Read assignment right to left, out loud, every time.

Objects, Strings and the dot operator

An object variable does not hold the data — it holds a **reference** to where the data lives. To reach inside, use the `.` (dot) operator.

```
String s = "Hello World";  
  
System.out.println(s);           // Hello World  
System.out.println(s.length());  // 11
```

- String values are surrounded by " marks — **the quotes are not part of the value**
- `s.length()` calls the `length()` method on `s`

Count the characters in "Hello World". The space counts.

You have been using this all along

```
System.out.println("Hello 41039!");
```

Now read it again, with the dot operator in mind:

- `System` — a class
- `.out` — a variable inside that class
- `.println(...)` — a method called on `out`

There is a version of `println` for each type, which is why it happily prints a `String`, an `int` or a `boolean`.

The magic incantation was three ordinary things all along.

What if you forget to initialise?

```
String s;  
  
System.out.println(s);  
System.out.println(s.length());
```

- Java refuses to compile this: error: variable `s` might not have been initialized
- Java is strict about local variables — it protects you here
- It will *not* always protect you. In other places an uninitialised object variable gets the special value `null` — a reference pointing at nothing

You will meet `NullPointerException` later. Not today.

Comments in Java

```
int i = 0;    // from here to the end of the line is ignored

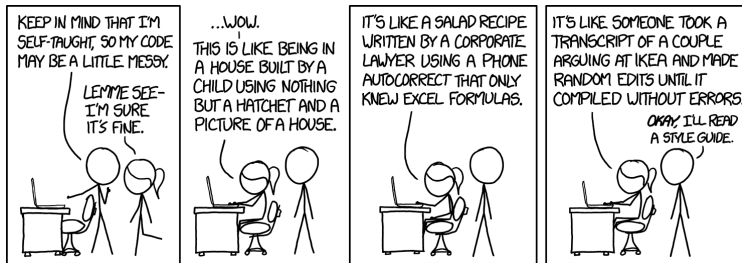
/* This is a multiline comment.

   You can write anything in here, including code:
   System.out.println(i);
   It will all be ignored by the compiler.
*/
```

- Comments are notes for humans. The compiler ignores them completely.
- Documenting your code is **assessed** in this subject (SLO4)
- Practical use today: comment a line *out* instead of deleting it while debugging

Shortcut in Ed: ctrl + / toggles comments on a line or selection.

Naming things



xkcd 1513, *Code Quality* — Randall Munroe, CC BY-NC 2.5 — xkcd.com/1513

“There are 2 hard problems in computer science: cache invalidation, naming things, and off-by-1 errors.”

Leon Bambrick, after Phil Karlton

Name a variable for the person reading it in Week 11. That person is you.

Outline

- 1 What Programming Is
- 2 Your First Java Program
- 3 Variables, Types and Values
- 4 The Same Ideas in Python
- 5 When Things Go Wrong
- 6 Ed, Labs and Getting Help

Python, the language

	Java	Python
Object oriented	yes	yes
Procedural	yes	yes
Imperative	yes	yes
Garbage collected	yes	yes
Execution	compiled	interpreted
Typing	static	dynamic

The lists are almost identical. **Only the last two rows differ** — and they explain nearly everything else you will notice.

Python is older than Java. 1991 versus 1995.

Hello, Python

```
print("Hello 41039!")
```

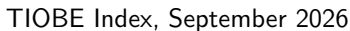
That's it. One line.

Why? Because **Python allows code outside of classes**. Java requires everything to live inside one.

- No ; at the end of the line
- No class, no main, no type declarations

Java's extra ceremony is not pointless — it buys the ability to detect certain errors *before* the program runs. The two languages made different bets.

Neither is better. This is a design trade-off, and you will keep meeting it.



1.	Python	17.76%
2.	C	10.28%
3.	C++	8.67%
4.	Java	7.54%

We teach **both**, deliberately.

xkcd 353, *Python* — Randall Munroe, CC BY-NC 2.5 — xkcd.com/353

Python shows the idea. Java shows what the ceremony buys.

Python's basic types

- str — text
- int — whole numbers, **one type, no size limit**
- float — decimals
- bool — True or False
- list — the everyday sequence type
- dict — key/value lookup

In Python, **variables** do not have a type — but the **data** still does.

Two traps

- Java has four integer types and you pick the size. Python has **one**.
- True/False are capitalised in Python; true/false are lowercase in Java.

That single difference is what "dynamically typed" means.

Indentation is not decoration

Java delimits blocks with { and }, so formatting is cosmetic. **Python uses the indentation itself.**

```
a = input("Enter a number: ")
if int(a) < 10:
    print("It's less than 10! Woo.")
```

```
a = input("Enter a number: ")
if int(a) < 10:
    pass
print("It's less than 10!")
```

The second version prints *always*. Only the indentation changed.

In Java bad indentation is ugly. In Python it is wrong.

Indentation: practical advice

- Use **spaces**, four of them, and let the editor do it
- **Never mix tabs and spaces** — it produces errors that are invisible on screen
- Python has no empty blocks. If you need a block that does nothing, use `pass`

Comments in Python

```
# This is a comment  
a = "This is not a comment" # This is, however.
```

A `#` inside a string is just a character. Python has no `/* */` equivalent.

Outline

- 1 What Programming Is
- 2 Your First Java Program
- 3 Variables, Types and Values
- 4 The Same Ideas in Python
- 5 When Things Go Wrong
- 6 Ed, Labs and Getting Help

Three kinds of error

- **Compile-time errors** — the compiler cannot compile your code. *It tells you.*
- **Run-time errors** — the code compiles, then crashes while running. *It tells you.*
- **Logic errors** — it runs perfectly and does the *wrong thing*. *Nothing tells you.*

You only get error messages for the first two. The third is the dangerous one, precisely because it is silent.

A QA engineer walks into a bar. Orders a beer. Orders 0 beers. Orders -1 beers. Orders a lizard.

The first real customer asks where the toilet is. The bar bursts into flames.

Programmer folklore, after Bill Sempf

Bugs are an unavoidable part of programming, not a sign you are bad at this.

8.5 million computers, one array index

IN THE NEWS

The code expected 20 values. The update sent 21.

CrowdStrike “Channel File 291” root cause analysis, reported by ABC News, 7 August 2024

Reading that 21st value ran off the end of the array. Every machine crashed — airlines, hospitals, banks.

It compiled. It shipped. **It passed their tests.**

Nothing told them. That is what a logic error looks like at scale.

Reading a Java compile-time error

```
<class>:<line>: error: <description>  
<problematic code>  
      ^
```

```
public class WontCompile {  
    public static void main(String[] args) {  
        System.out.println("Hi " + userName + " how are you?");  
    }  
}
```

```
WontCompile.java:3: error: cannot find symbol  
    System.out.println("Hi " + userName + " how are you?");  
                        ^
```

Which file → which line → what kind → which token (follow the ^).

Two things that will save you

1. Turn on Line Numbers in the Ed editor

Without them, `WontCompile.java:3` tells you nothing.

2. Fix the first error, then recompile

- One small bug often produces *many* error messages
- The further your code is from valid Java, the more confused the compiler gets
- Errors 2 through 14 are usually downstream of error 1

“cannot find symbol” means *you used a name I have never seen* — almost always a typo, a missing declaration, or a capitalisation slip.

Do not read all fourteen errors. Fix one, run again.

Java run-time errors: the stack trace

```
Exception in thread "<thread>" <errorClass>: <errorMessage>
    at <class>.<method>(<file>:<lineNumber>)
    at <class>.<method>(<file>:<lineNumber>)
    ...
```

- Ignore thread entirely for this subject
- `errorClass` is the *type* of error — you can look this up
- The remaining lines are the **stack trace**: the first is where it crashed, the second is what called that, and so on

A crash happens *in* one class but may be *caused* by another, several steps removed. The last person holding the broken thing is not always who broke it.

Strategy: find your own code in the stack trace.

A real one

```
Scanner input = new Scanner(System.in);  
System.out.println("Hi, what's your name?");  
int userName = input.nextInt();
```

```
Exception in thread "main" java.util.InputMismatchException  
    at java.base/java.util.Scanner.throwFor(Scanner.java:943)  
    at java.base/java.util.Scanner.next(Scanner.java:1598)  
    at java.base/java.util.Scanner.nextInt(Scanner.java:2263)  
    at java.base/java.util.Scanner.nextInt(Scanner.java:2217)  
    at HelloAgain.main(HelloAgain.java:10)
```

Four Scanner frames — a thoroughly tested library, probably not the culprit. One HelloAgain.main frame. **That one is yours.** Line 10 reads a name into an int.

This program works fine if your name is a number.

Errors in Python

```
Traceback (most recent call last):  
  File "<filename>", line <linenumber>, in <module>  
    <line of code that caused the problem>  
<Type of Error>: <description of error>
```

```
n = "This is not a number"  
  
print(n + 1)
```

- TypeError — + cannot be applied to a str and an int
- Python puts the error type **last**; Java puts it **first**

In a Python traceback, read the last line first.

The same +, two languages

Java

```
int c = 56;  
System.out.println("c = " + c);
```

Prints `c = 56`. Java converts the `int` to a `String` and joins them.

Python

```
n = "This is not a number"  
print(n + 1)
```

`TypeError`. Python refuses.

Same operator, same shape, opposite outcome. This is what **static versus dynamic typing** looks like in practice — not an abstraction, a visible consequence.

Fix the Python one so it prints: This is not a number1

Debugging, honestly



xkcd 1739, *Fixing Problems* — Randall
Munroe, CC BY-NC 2.5 —
xkcd.com/1739

“Debugging is twice as hard as writing a program in the first place. So if you’re as clever as you can be when you write it, how will you ever debug it?”

Brian Kernighan, *The Elements of Programming Style*, 1978

99 little bugs in the code. Take one down, patch it around — **127** little bugs in the code.

Programmer folklore

Clever code is a loan. Debugging is when it comes due.

Outline

- 1 What Programming Is
- 2 Your First Java Program
- 3 Variables, Types and Values
- 4 The Same Ideas in Python
- 5 When Things Go Wrong
- 6 Ed, Labs and Getting Help

Lessons page

- Work through it **top to bottom**
- Lectures, readings, coding exercises
- Readings are reference material — you will come back to them all session

Discussion board

- Search first
- **New thread**, short descriptive title
- **Public** unless genuinely sensitive
- Anonymous hides you from students, **not from staff**
- **Do not paste code** — we can already see your submissions

Ed provides Java and Python in the browser. Nothing needs installing.

Why we ask you to post publicly



xkcd 979, *Wisdom of the Ancients* — Randall Munroe,
CC BY-NC 2.5 — xkcd.com/979

- A **public** thread answers it once, for everybody
- A private one answers it once, for you
- **Search first**

Solved it yourself after posting? **Reply with the answer.**

Do not be the 2003 forum post that says “never mind, fixed it”.

Run and Mark

Run

Watch your code behave, and interact with it.

Mark

Automated tests: compile, run, feed inputs, check output.

“But it works on my machine!”

Ed is the machine that counts.

Programmer folklore

Three results

- **Passed** — move on
- **Not yet** — it ran, output was wrong (a logic error)
- **Errored** — it did not compile

- **Unlimited attempts** — attempt 1 and attempt 101 score the same
- **Output must match exactly** — a stray space fails the test

Run until it behaves. Then Mark.

Labs

- **Lab classes start in Week 2.** There is no lab this week.
- 1.5 hours: practice exercises with the tutor, then assessed exercises with one-on-one help
- **Practice** exercises are not assessed and have no deadline. Do them first.
- **Assessed** exercises are due **11:55pm on the Monday of the following week**
- You must pass **every** test case in an exercise to get marks for it

Prepare by working through the lessons before the lab, and glancing at the exercises. Ask questions well before the deadline — the board is not monitored over the weekend.

Get stuck on Friday, not Sunday night.

One thing to be clear about

Learning to code is tough. You **will** get stuck. Everyone does.

This is good practice

- Looking up an error message
- Reading documentation
- Asking on Ed, or in a lab
- Learning a concept from anywhere

This is misconduct

- Pasting in a solution you cannot explain
- Copying from a classmate

Similarity tools analyse **how code works**, not how it looks — renaming variables does not help. And four times this session you will be asked to explain code out loud, in person.

Copied code does not survive that conversation. Ask for help instead.

Why we care that you can read code

IN THE NEWS

84% of developers use AI. 3.1% highly trust what it gives back.

Stack Overflow Developer Survey 2025

Biggest frustration, **66%**: *“AI solutions that are almost right, but not quite.”*

“Almost right” is only visible **if you can read the code.**

The industry’s biggest complaint about AI is a reading problem.

This week

On Ed, work through all seven Week 1 lessons:

- 1 Everything you need to know about Ed — *turn on Line Numbers*
- 2 Everything you need to know about labs
- 3 An important warning
- 4 The Basics of Programming
- 5 Introduction to Java
- 6 Introduction to Python
- 7 Debugging Programs

Run the code in the readings. Change things. Break things on purpose and read what the error says. Refreshing the page restores the original — you cannot damage anything.

Next week: variables, types, and talking to the user.

Sources and credits

Articles

- METR, *Early-2025 AI and Experienced Open-Source Developer Productivity*, Jul 2025 — metr.org
- OpenJDK, *JEP 512* — openjdk.org/jeps/512
- The Register, *Gangnam Style BREAKS YouTube*, Dec 2014 — theregister.com
- TIOBE Index, Sep 2026 — tiobe.com/tiobe-index/
- ABC News, *CrowdStrike root cause analysis*, Aug 2024 — abc.net.au
- *Stack Overflow Developer Survey 2025* — survey.stackoverflow.co/2025/ai

Comics

All by **Randall Munroe**, xkcd.com, used under **CC BY-NC 2.5** — 303, 353, 979, 1425, 1513, 1739, 1838.

Check a claim yourself. That habit is worth more than any slide here.

Questions?



Dr. Xuan-Bach Le · le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT